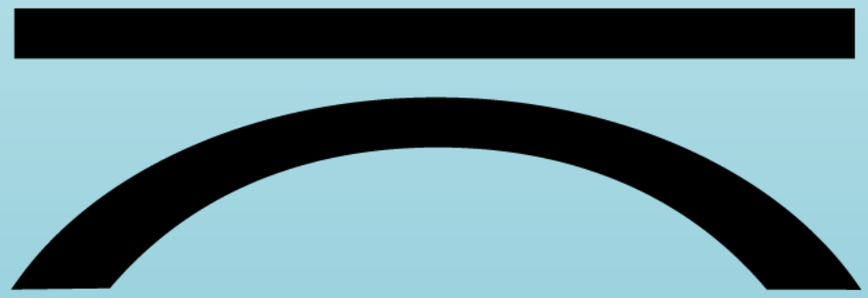
CISP 400

Introduction to Structured Programming

Dr. Fowler



FOLSOM LAKE COLLEGE

EL DORADO CENTER | RANCHO CORDOVA CENTER

Agenda
Ch 10 - Pointers

C++ Memory Model

10.1 Pointers and the Address Operator

10.2 Pointer Variables

10.3 The Relationship between Arrays and pointers

10.4 Pointer Arithmetic

10.5 Initializing Pointers

10.6 Comparing Pointers

10.7 Pointers as Function Parameters

10. 8 Pointers to Constants and Constant Pointers

10.9 Dynamic Memory Allocation

10.10 Returning Pointers from Functions

10.11 Pointers to Class Objects and Structures

10.12 Selecting Members of Objects

10.13 Smart Pointers

Agenda
Ch 10 - Pointers

C++ Memory Model

10.1 Pointers and the Address Operator

10.2 Pointer Variables

10.3 The Relationship between Arrays and pointers

10.4 Pointer Arithmetic

10.5 Initializing Pointers

10.6 Comparing Pointers

10.7 Pointers as Function Parameters

10. 8 Pointers to Constants and Constant Pointers

10.9 Dynamic Memory Allocation

10.10 Returning Pointers from Functions

~~10.11 Pointers to Class Objects and Structures~~

~~10.12 Selecting Members of Objects~~

10.13 Smart Pointers

Big Idea

- We can directly access memory objects.

C++ allows us to remove the abstraction of the symbol table and access memory from the program - directly.

Consider

- It’s easier to give someone your house address than a copy of your house.

Advantages

- Speed - closer to the hardware.
- Efficient - don’t copy large memory objects.
- C makes extensive use of them.
- Access to the heap.

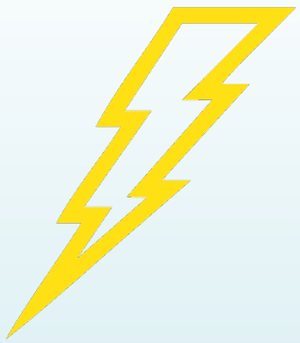
Advantages

- Speed - closer to the hardware.
- Efficient - don’t copy large memory objects.
- C makes extensive use of them.
- **Access to the heap.**

Objective for Today

Today, we are going to use the heap to overcome one of the biggest limitations of arrays - fixed size.

Lightning Review

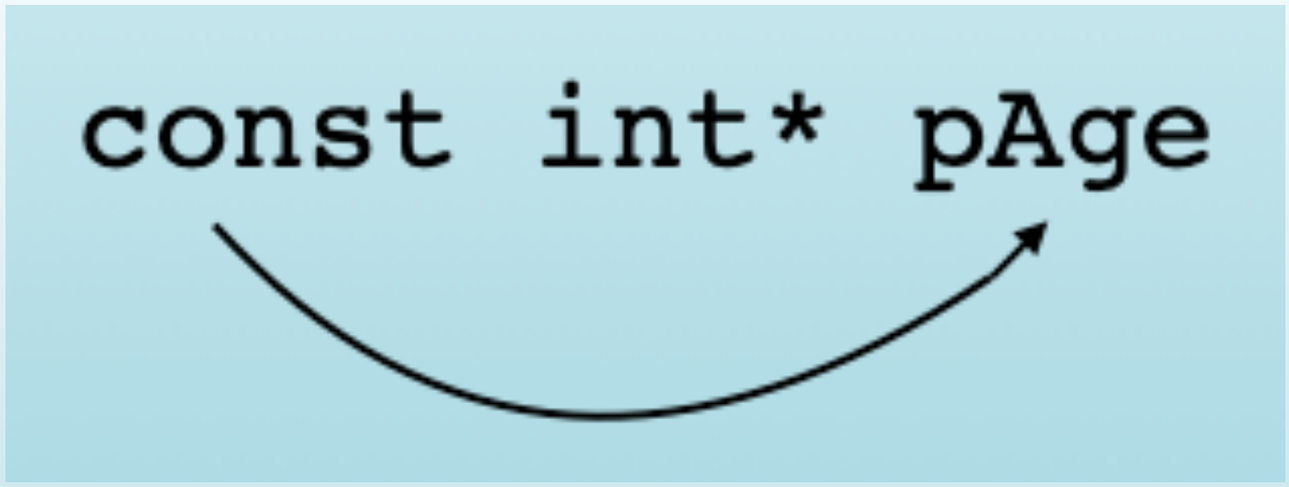
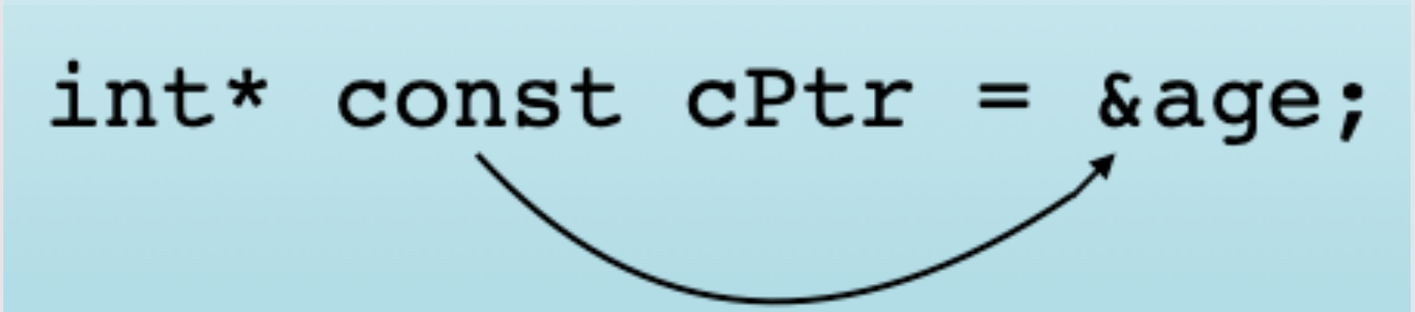


Lightning Review Questions

- How would it be possible to use const with a pointer variable?

- Can we use pointer variable as arguments/ parameters?

- What other questions to you have for me?

Pointer to a Constant		data immutable
Constant Pointer		address immutable
Constant Pointer to a Constant		data and address immutable

Sending a pointer variable

```
 1 // 10-11.ccp -- Pointers as function parameters
 2 #include <iostream>
 3 using namespace std;
 4
 5 void getNumber(int*);
 6 void doubleValue(int*);
 7
 8 int main()
 9 {
10   int number;
11   getNumber(&number);
12   doubleValue(&number);
13   cout << "Value doubled is " << number << endl;
14 }
15
16 void getNumber(int* input)
17 {
18   cout << "Enter an integer number: ";
19   cin >> *input;
20 }
21
22 void doubleValue(int* val)
23 {
24   *val *= 2;
25 }
```

Returning a pointer variable

```cpp
// 10-15.cpp - returning a pointer from a function
#include <iostream>
#include <cstdlib>      // rand and srand
#include <ctime>        // time function
#include <assert.h>
//#define NDBUG
using namespace std;

// Function Prototypes
int* getRandomNumbers(int);

int main()
{
   int* numbers = nullptr; // point to the numbers

   numbers = getRandomNumbers(5);  // build the array

   for (int count = 0; count < 5; count++)
   cout << numbers[count] << endl;

   delete [] numbers;
   numbers = nullptr;
   return 0;
}

int* getRandomNumbers(int size)
{
   assert(size > 0);       // protect our function
   int* array = nullptr;   // array to hold numbers

   array = new int[size];

   srand(time(0));     // seed with time

   for (int count = 0; count < size; count++)
   array[count] = rand();
   return array;
}
```

Exercise

Desk Check

Exercise Goals

- Review of code syntax

- Check understanding of syntax

Desk Check Exercise

- Pair up
- Manually determine the output of the following code?
  DO NOT TYPE THIS IN AND EXECUTE IT.
- We'll spend about 10 minutes on this, then go over it.

```cpp
//output10-2.cpp - What's the Output?
#include <iostream>
using namespace std;

int main() {
  int x = 5, y = 10, z = 15;
  int* ptrX = &x;
  int* ptrY = &y;

  *ptrX = z;
  ptrY = ptrX;
  *ptrY = 7;

  cout << x << " " << y << " " << z << endl;
  cout << *ptrX << " " << *ptrY << endl;

  return 0;
}
```

```cpp
//output10-2.cpp - What's the Output?
#include <iostream>
using namespace std;

int main() {
    int x = 5, y = 10, z = 15;
    int* ptrX = &x;
    int* ptrY = &y;

    *ptrX = z;
    ptrY = ptrX;
    *ptrY = 7;

    cout << x << " " << y << " " << z << endl;
    cout << *ptrX << " " << *ptrY << endl;

    return 0;
}
```

Run

```
7 10 15
7 7
```

10.9 Dynamic Memory Allocation

Problem with Arrays

Static structures - once defined, we cannot change the size.

How to get around this?

- The problem is our variables are stored on the stack.
- This is a fixed size data structure - can’t change.

- We know there is a pool of memory we can access which is unaffected by the stack.

Access to the Heap

```
new()

delete
delete []
```

delete and delete []

- delete deallocates memory for a single object created with new.

- delete [] deallocates memory for an array created with new [].

```cpp
// new.cpp -- introduction to new()
#include <iostream>
#include <string>
using namespace std;

int* MakeHeap();
void ShowHeap(string, int*);

int main() {
  int* pInt = nullptr;
  pInt = MakeHeap();

  ShowHeap("Data in new heap variable: ", pInt);

  *pInt = 5;
  ShowHeap("Updated data in new heap variable: ", pInt);

  return 0;
}

int* MakeHeap() {
  int* pTmp = nullptr;
  pTmp = new int;
  return pTmp;
}


void ShowHeap(string msg, int* pTmp) {
  cout << msg << *pTmp << endl;

}
```

```cpp
// new.cpp -- introduction to new()
#include <iostream>
#include <string>
using namespace std;

int* MakeHeap();
void ShowHeap(string, int*);

int main() {
  int* pInt = nullptr;
  pInt = MakeHeap();

  ShowHeap("Data in new heap variable: ", pInt);

  *pInt = 5;
  ShowHeap("Updated data in new heap variable: ", pInt);

  return 0;
}

int* MakeHeap() {
  int* pTmp = nullptr;
  pTmp = new int;
  return pTmp;
}


void ShowHeap(string msg, int* pTmp) {
  cout << msg << *pTmp << endl;

}
```

```
> ./main
Data in new heap variable: 0
Updated data in new heap variable: 5
>
```

```cpp
// new2.cpp -- introduction to new()
#include <iostream>
#include <string>
using namespace std;

int* MakeHeap(int);
void ShowHeap(string, int*);

int main() {
  int* pInt = nullptr;
  pInt = MakeHeap(10);

  ShowHeap("Data in new heap variable: ", pInt);

//  *pInt = 5;  // now this won't work.
  delete pInt;
  pInt = nullptr;
  pInt = MakeHeap(50);

  ShowHeap("Updated data in new heap variable: ", pInt);

  return 0;
}

// This now returns a pointer to a constant
int* MakeHeap(int n) {
  int* pTmp = nullptr;
  pTmp = new int{n};
  int* const cPtr = pTmp;
  return cPtr;
}


void ShowHeap(string msg, int* pTmp) {
  cout << msg << *pTmp << endl;

}
```

```cpp
// new2.cpp -- introduction to new()
#include <iostream>
#include <string>
using namespace std;

int* MakeHeap(int);
void ShowHeap(string, int*);

int main() {
  int* pInt = nullptr;
  pInt = MakeHeap(10);

  ShowHeap("Data in new heap variable: ", pInt);

//  *pInt = 5;  // now this won't work.
  delete pInt;
  pInt = nullptr;
  pInt = MakeHeap(50);

  ShowHeap("Updated data in new heap variable: ", pInt);

  return 0;
}

// This now returns a pointer to a constant
int* MakeHeap(int n) {
  int* pTmp = nullptr;
  pTmp = new int{n};
  int* const cPtr = pTmp;
  return cPtr;
}


void ShowHeap(string msg, int* pTmp) {
  cout << msg << *pTmp << endl;

}
```

```
> ./main
Data in new heap variable: 10
Updated data in new heap variable: 50
```

- When we use a scalar variable and a pointer variable, we have two ways to access the variable's data.
    - via the variable name
    - via the pointer variable
- We can have the pointer variable point to somewhere else, but the regular variable is still there.

- Contrast that with heap variables (created with new).

- These are ONLY accessible with a pointer.

- If we lose the address we lose the variable.

- We also have the ability to destroy a variable (why keep a password around once authenticated?).

Advantages of Heap Variables

- Resizable arrays
- Faster access for larger memory objects.
- More efficient use of memory.
- Deleteable variables.
- Access to objects outside of current scope.

But ....

This still isn’t good enough.

Can we make this work with an array?

Pseudo Dynamic Arrays

- Arrays are created on the stack by default.

- We can also create them on the heap.

- This allows the size of the array to be determined at RUNTIME and not compile time!

```cpp
// 10-14.cpp Pseudo Dynamic Memory
#include <iostream>
#include <iomanip>
using namespace std;

int main(){
    double* sales = nullptr,    // set to no address
        total = 0.0,              // accumulator
        average;                    // Average sales
    int numDays;                  // number days sales

    // Get the number of days sales
    cout << "How many days of sales figures do you wish ";
    cout << "to process? ";
    cin >> numDays;

    // Dynamically allocate array - 1 time
    sales = new double[numDays];    // allocate memory

    // Get sales for each day
    cout << "Enter the sales figures below.\n";
    for (int count = 0; count < numDays; count++){
        cout << "Day " << (count + 1) << ": ";
        cin >> sales[count];
    }

    // Calculate total sales
    for (int count = 0; count < numDays; count++)
        total += sales[count];
    average = total / numDays;  // Calculate average sales per day

    // Display
    cout << setprecision(2) << fixed << showpoint;
    cout << "\n\nTotal Sales: $" << total << endl;
    cout << "Average Sales: $" << average << endl;

    // Free Memory!
    delete[] sales;
    sales = nullptr;
    return 0;
}
```

This is a step closer, but not there yet.

Algorithm to create pseudo-dynamic arrays

1. Declare a pointer variable (#7).

2. Prompt the user for the array size.

3. Allocate the memory using (1) and (2) and the **new** command (#18).

    - Note: We can manipulate our pointer variable just like a regular array variable.

4. When done, use **delete** to free memory and set the pointer variable to **nullptr** (#38).

Let’s put what we just learned to make a real dynamic array.

Exercise

Desk Check

Exercise Goals

- Review of code syntax

- Check understanding of syntax

Desk Check Exercise

- Pair up
- Manually determine the output of the following code?
  DO NOT TYPE THIS IN AND EXECUTE IT.

- We'll spend about 10 minutes on this, then go over it.

```cpp
// output10-3.cpp - What's the Output?
#include <iostream>
using namespace std;

int *UpdatePtr(int *, int);
void ModifyValue(int *);

int main() {
  int *ptrA = nullptr;
  int value = 15;

  ptrA = UpdatePtr(ptrA, value);
  ModifyValue(ptrA);

  cout << *ptrA << " " << ptrA << endl;

  delete ptrA;
  return 0;
}

int *UpdatePtr(int *p, int val) {
  p = new int;
  *p = val;
  return p;
}

void ModifyValue(int *p) { *p += 10; }
```

```cpp
//output10-3.cpp - What's the Output?
...

int* UpdatePtr(int*, int);
void ModifyValue(int*);

int main() {
  int* ptrA = nullptr;
  int value = 15;

  ptrA = UpdatePtr(ptrA, value);  // Allocate memory and assign value
  ModifyValue(ptrA);              // Modify the value of the allocated memory

  cout << *ptrA << " " << ptrA << endl;  // Print the value and address

  delete ptrA;   // Free the dynamically allocated memory
  return 0;
}

int* UpdatePtr(int* p, int val) {
  p = new int;    // Allocate memory for an integer
  *p = val;       // Assign value to the allocated memory
  return p;
}

void ModifyValue(int* p) {
  *p += 10;  // Add 10 to the current value
}
```

Expected Output:
25 <address of ptrA>

How do we create a real dynamic array?

```cpp
// rdaa.cpp  -- real dynamically allocated array
#include <iostream>
using namespace std;

int main(){     // build the initial array
    int* pInt = nullptr;
    int size = 1;

    pInt = new int[size];
    pInt[0] = 10;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

// increase by 1
    int* pTemp = nullptr;
    size++;
    pTemp = new int[size];

    pTemp[0] = pInt[0];
    pTemp[1] = 20;

    delete[] pInt;
    pInt = pTemp;
    pTemp = nullptr;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

    // decrease by 1 from the right
//    int* pTemp = nullptr;
    size--;
    pTemp = new int[size];
    pTemp[0] = pInt[0];

    delete[] pInt;
    pInt = pTemp;
    pTemp = nullptr;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

    return 0;
}
```

```cpp
// rdaa.cpp  -- real dynamically allocated array
#include <iostream>
using namespace std;

int main(){     // build the initial array
    int* pInt = nullptr;
    int size = 1;

    pInt = new int[size];
    pInt[0] = 10;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

// increase by 1
    int* pTemp = nullptr;
    size++;
    pTemp = new int[size];

    pTemp[0] = pInt[0];
    pTemp[1] = 20;

    delete[] pInt;
    pInt = pTemp;
    pTemp = nullptr;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

    // decrease by 1 from the right
//    int* pTemp = nullptr;
    size--;
    pTemp = new int[size];
    pTemp[0] = pInt[0];

    delete[] pInt;
    pInt = pTemp;
    pTemp = nullptr;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

    return 0;
}
```

```
10
10  20
10
```

The Big Idea

- Build your data structure out on the heap.
- Build your larger/smaller replacement data structure out on the heap.
- Copy the data you want to save from the original data structure to the replacement data structure.
- Delete the original data structure and clean up any pointers.

Algorithm for a dynamic array

1. Create main array pointer variable (#6).

2. Set the array size (#7)

3. Allocate **new** memory on the heap (#9).

4. Put initial values in it.

Just like pseudo-dynamic array so far …

Algorithm for increasing a dynamic array

1. Increment our SIZE variable (#18)
2. Create a temporary array with the new SIZE (#17, 19)
3. Copy to old array values to the temp array. (#21)
4. Add the new value(s) to the temporary array. (#22)
5. Free the memory for the original array (#24).
6. Copy the pointer to the temp array to the original array pointer variable (#25).
7. Set the temp pointer to nullptr (#26).

```cpp
// rdaa.cpp  -- real dynamically allocated array
#include <iostream>
using namespace std;

int main(){     // build the initial array
    int* pInt = nullptr;
    int size = 1;

    pInt = new int[size];
    pInt[0] = 10;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

// increase by 1
    int* pTemp = nullptr;
    size++;
    pTemp = new int[size];

    pTemp[0] = pInt[0];
    pTemp[1] = 20;

    delete[] pInt;
    pInt = pTemp;
    pTemp = nullptr;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

    // decrease by 1 from the right
//    int* pTemp = nullptr;...
}
```

Algorithm for decreasing a dynamic array

1. Increment our SIZE variable (#34)

2. Create a temporary array with the new SIZE (#35)

3. Copy to old array values to the temp array (#36). Don't copy over the values you want to remove from the new array.

4. Free the memory for the original array (#38).

5. Copy the pointer to the temp array to the original array pointer variable (#39).

6. Set the temp pointer to nullptr (#40).

```cpp
// rdaa.cpp  -- real dynamically allocated array
#include <iostream>
using namespace std;

int main(){     // build the initial array
    int* pInt = nullptr;
    int size = 1;

    pInt = new int[size];
    pInt[0] = 10;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

// increase by 1...
//    int* pTemp = nullptr;
    size--;
    pTemp = new int[size];
    pTemp[0] = pInt[0];

    delete[] pInt;
    pInt = pTemp;
    pTemp = nullptr;

    for (int i = 0; i < size; i++)
    cout << pInt[i] << "  ";
    cout << endl;

    return 0;
}
```

Program This

Exercise

Exercise Goals

- Increase Topic Mastery
- Construct Knowledge
- Give you data to evaluate how well you are learning.

Programming Exercise

- Build a dynamic array of 10 ints on the heap. Assign a value of 10 * index number to each element.
- Display the array.
- Wrap the following in a 5 count loop:
  - Generate a random number. Even - add 1 element to the array. Odd -subtract 1 element from the array.
  - Print what happened.
- Print the entire loop again.

What's this section about?

Agenda
Ch 10 - Pointers

C++ Memory Model

10.1 Pointers and the Address Operator

10.2 Pointer Variables

10.3 The Relationship between Arrays and pointers

10.4 Pointer Arithmetic

10.5 Initializing Pointers

10.6 Comparing Pointers

10.7 Pointers as Function Parameters

10. 8 Pointers to Constants and Constant Pointers

10.9 Dynamic Memory Allocation

10.10 Returning Pointers from Functions

~~10.11 Pointers to Class Objects and Structures~~

~~10.12 Selecting Members of Objects~~

10.13 Smart Pointers

Big Idea

- We can directly access memory objects.

C++ allows us to remove the abstraction of the symbol table and access memory from the program - directly.

Consider

- It’s easier to give someone your house address than a copy of your house.

Advantages

- Speed - closer to the hardware.
- Efficient - don’t copy large memory objects.
- C makes extensive use of them.
- Access to the heap.

Advantages

- Speed - closer to the hardware.
- Efficient - don’t copy large memory objects.
- C makes extensive use of them.
- **Access to the heap.**

Objective for Today

Today, we are going to use the heap to overcome one of the biggest limitations of arrays - fixed size.


NEXT TIME

Agenda
Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

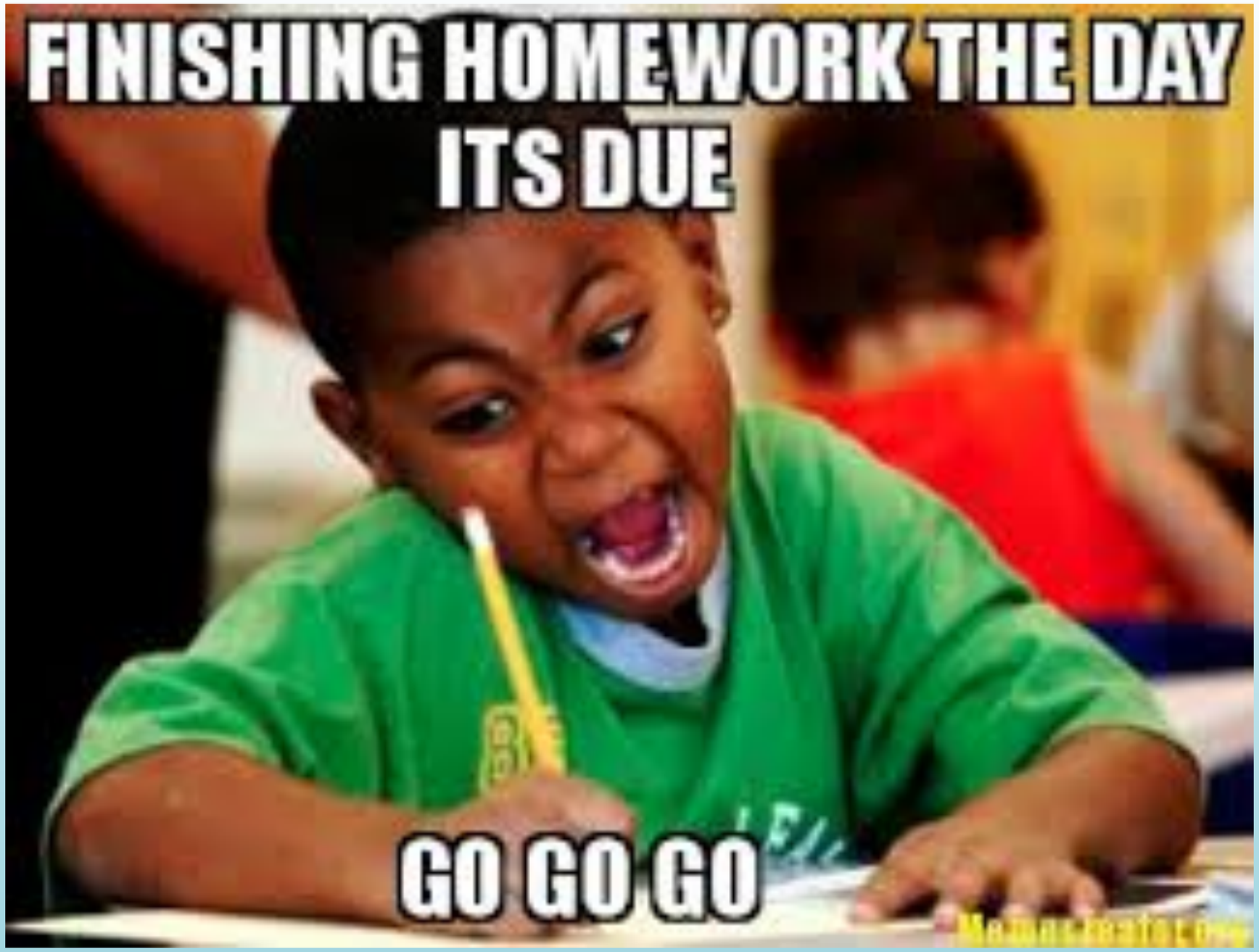
7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data Types

8.13 Arrays of Objects



Hw1 Peer Review due Sunday.

No Class Next Week

Exercise

Desk Check

Exercise Goals

- Review of code syntax

- Check understanding of syntax

Desk Check Exercise

- Pair up
- Manually determine the output of the following code?
  DO NOT TYPE THIS IN AND EXECUTE IT.

- We'll spend about 10 minutes on this, then go over it.

```cpp
// output1209.cpp -- What's the output?
#include <iostream>
using namespace std;

int main() {
    double dec1 = 2.5;
    double dec2 = 3.8;
    double* p;
    double* q;

    p = &dec1;
    *p = dec2 - dec1;

    q = p;
    *q = 10.0;

    *p = 2 * dec1 + (*q);
    q = &dec2;
    dec1 = *p + *q;

    cout << dec1 << " " << dec2 << endl;
    cout << *p << " " << *q << endl;
}
```

```cpp
// output1209.cpp -- What's the output?
#include <iostream>
using namespace std;

int main() {
    double dec1 = 2.5;
    double dec2 = 3.8;
    double* p;
    double* q;

    p = &dec1;
    *p = dec2 - dec1;

    q = p;
    *q = 10.0;

    *p = 2 * dec1 + (*q);
    q = &dec2;
    dec1 = *p + *q;

    cout << dec1 << " " << dec2 << endl;
    cout << *p << " " << *q << endl;
}
```

```
gcc version 4.6.3
>
33.8 3.8
33.8 3.8
>
```

Homework 1 Peer Review

Exercise

Hw Debrief

Increases topic mastery.

Construct knowledge.

Process

- Pair up with group ?
- Review each other's homework.
- Complete the Review Sheet.
- Exchange sheets.
- Reflect and comment on the feedback you received.
- Turn it in.

Debrief

What worked about this exercise?

What could be improved about this exercise?

Why did I ask you to do ….

Connect activity with course objectives/SLO’s.

Connect activity with course readings.